

텍스트 분석

✓ 0. 텍스트 분석과 NLP

📌 NLP와 텍스트 분석의 차이

- NLP (Natural Language Processing): 머신이 인간의 언어를 이해하고 해석하는 데 중점을 두는 기술. 예: 기계 번역, 질의응답 시스템 등
- 텍스트 분석 (Text Analytics, TA): 비정형 텍스트에서 의미 있는 정보를 추출하는 데 중점을 둬. 텍스트 마이닝(Text Mining)이라고도 함.

📌 머신러닝과 텍스트 분석

- 과거에는 규칙 기반(rule-based)으로 텍스트를 분석했지만, 현재는 머신러닝 기반의 모델이 텍스트 데이터를 학습하고 예측하는 구조로 발전함. 그 결과 더 정교한 분석이 가능해짐.

📌 텍스트 분석 주요 기술 분야

1. 텍스트 분류 (Text Classification / Categorization)

- 문서를 특정 카테고리로 자동 분류하는 기법.
- 예: 뉴스 기사 분류, 스팸 메일 필터링 등
- 지도학습 기반

2. 감성 분석 (Sentiment Analysis)

- 텍스트에서 감정, 의견, 판단, 태도 등을 추출하는 기법
- 예: SNS 감정 분석, 영화 리뷰 긍정/부정 판단
- 지도학습과 비지도학습 모두 가능
- 텍스트 분석 분야에서 가장 활발하게 활용됨

3. 텍스트 요약 (Summarization)

- 긴 텍스트에서 핵심 주제나 중심 내용을 추출
- 대표 기술: 토픽 모델링 (Topic Modeling)

4. 텍스트 군집화 및 유사도 측정

- 유사한 문서를 비지도학습 기반으로 자동 군집화

- 문서 간의 유사도를 측정해 비슷한 문서를 분류하는 데 활용
-

✓ 1. 텍스트 분석 이해

📌 텍스트 분석이란?

- 비정형 텍스트 데이터를 분석하는 기법.
- 머신러닝 모델은 일반적으로 숫자형 정형 데이터만 입력할 수 있으므로,
- 텍스트를 벡터 형태의 피처로 변환하는 작업이 선행되어야 함.

📌 텍스트 → 숫자형 피처 변환 (Feature Vectorization)

- 텍스트를 단어(Word) 또는 단어 일부 기반 피처로 나눈 뒤,
각 피처에 숫자 값 (예: 단어 빈도수) 을 부여하여 벡터로 표현.
- 이 과정을 피처 벡터화(Feature Vectorization) 또는 피처 추출(Feature Extraction) 이라고 함.
- 대표 기법:
 - BOW (Bag of Words)
 - Count 기반
 - TF-IDF 기반
 - Word2Vec

📌 텍스트 분석 프로세스

1. 텍스트 전처리 (사전 준비 작업)
 - 클렌징 (불필요한 기호, 특수문자 제거)
 - 대/소문자 통일
 - 토큰화 (Tokenization)
 - 불용어 제거 (Stop Words)
 - 어근 처리 (Stemming / Lemmatization)
2. 피처 벡터화 / 추출
 - 전처리된 텍스트 → 수치 벡터로 변환
 - 예: BOW (Count, TF-IDF), Word2Vec 등

3. ML 모델 수립 및 학습/예측/평가

- 벡터화된 데이터를 기반으로 머신러닝 모델 적용

파이썬 기반 주요 텍스트 분석 라이브러리

라이브러리	특징
NLTK	Python 대표 NLP 패키지. 광범위한 기능 제공. 다만, 속도와 실무 활용성 면에서는 아쉬움
Gensim	토픽 모델링 특화. Word2Vec 기능도 강력
SpaCy	최신 기술, 빠른 처리 속도. 실무에서 가장 활발히 사용
Scikit-Learn	머신러닝 특화. NLP 전용 기능은 부족하지만, 벡터화와 분류 등 텍스트 기반 ML 적용 가능

2. 텍스트 사전 준비 작업(텍스트 전처리) - 텍스트 정규화

주요 전처리 작업

1. 클렌징 (Cleansing)

- 분석에 불필요한 기호, 특수문자, HTML/XML 태그 등 제거

2. 토큰화 (Tokenization)

텍스트를 작은 단위(문장 또는 단어)로 쪼개는 작업

- 문장 토큰화
 - 문장 단위로 분리
 - 함수: `nltk.sent_tokenize()`
- 단어 토큰화
 - 문장을 단어 단위로 분리
 - 함수: `nltk.word_tokenize()`
- n-gram
 - 연속된 n개 단어를 하나의 토큰으로 추출

3. 스톱워드 제거 (Stop Words Filtering)

- `is`, `the`, `a`, `in`, `you` 와 같은 의미 없는 단어 제거

- 너무 자주 등장해 의미가 흐려지는 단어들은 제외
- 함수: `nltk.corpus.stopwords.words('english')`

4. 어간 추출 (Stemming)

- 단어의 어근(root) 을 추출
- 단순 규칙 기반 (철자 손상 가능)
- 예: "working", "works", "worked" → "work"

5. 표제어 추출 (Lemmatization)

- 문법적/의미적 기준에 따라 원형 단어로 변환
- 철자 보존 + 더 정확함
- 품사 입력 필요 (`v` : 동사, `a` : 형용사 등)
- 함수: `WordNetLemmatizer().lemmatize(word, pos)`
- 예: "happiest" → "happy"

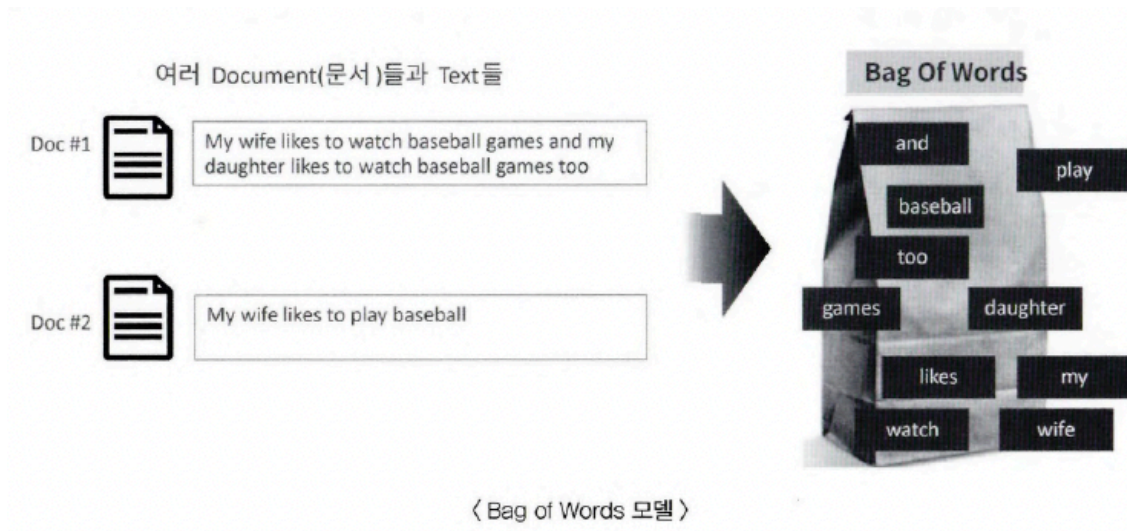
3. Bag of Words - BOW

- 문서 내 모든 단어의 순서나 문맥을 무시하고 단어의 등장 횟수를 기준으로 피쳐 (feature)를 생성하는 모델
- 모든 단어를 "봉투(Bag)"에 넣고 섞은 다음, 몇 번 등장했는지 수를 세는 방식
- 예:

문장 1: "My wife likes to watch baseball games"

문장 2: "My wife likes to play baseball"

→ 전체 단어를 나열하고 문장마다 등장 횟수를 기록 → 피쳐 벡터 생성



📌 BOW 모델의 장점과 단점

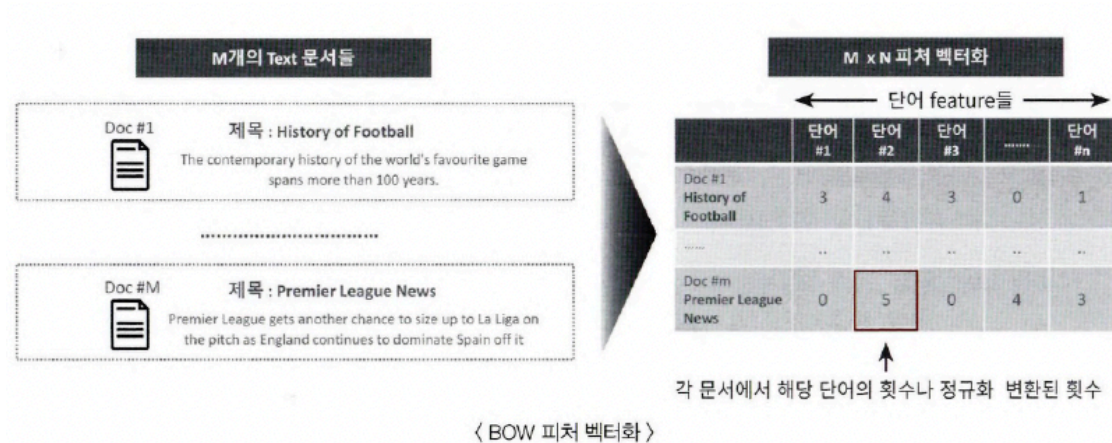
- 장점
 - 단순하고 빠르며, 특정 문서의 주요 단어 특징을 효과적으로 표현 가능
- 단점
 - 문맥 정보(semantic context) 반영 불가 → 단어 순서 상실
 - 희소 행렬(Sparse Matrix) 생성 → 대부분의 값이 0

📌 피처 벡터화

- 텍스트를 숫자 벡터로 변환하는 작업
- 텍스트를 단어 단위로 나누고, 각 단어에 빈도나 가중치 값을 부여
- 머신러닝 모델이 텍스트를 입력으로 처리할 수 있도록 함

	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6	Index 7	Index 8	Index 9	Index 10
	and	baseball	daughter	games	likes	my	play	to	too	watch	wife
문장 1	1	2	1	2	2	2		2	1	2	1
문장 2		1			1	1	1	1			1

→ 문장 1에서 baseball은 2회 나타남



📌 BOW의 벡터화 방식

1. 카운트 벡터화 (Count Vectorization)

- 단어가 문서에 등장한 횟수를 그대로 값으로 부여

2. TF-IDF 벡터화

- TF (Term Frequency): 문서 내 단어의 빈도
- IDF (Inverse Document Frequency): 전체 문서 중 해당 단어가 얼마나 희귀한지
- 자주 등장하지만 흔한 단어는 패널티, 특정 문서에만 자주 등장하는 단어는 가중치 부여

📌 사이킷런의 벡터화 도구

1. CountVectorizer

- 카운트 기반 피쳐 벡터화
- 전처리 + 토큰화 + 스톱워드 제거 가능
- 주요 파라미터:
 - `max_df`: 너무 자주 등장하는 단어 제거
 - `min_df`: 너무 드물게 등장하는 단어 제거
 - `ngram_range`: 단어 묶음 (예: (1,2) → unigram + bigram)

2. TfidfVectorizer

- TF-IDF 가중치를 부여하는 벡터화 방법
- `CountVectorizer` 와 유사한 파라미터 사용

📌 희소 행렬(Sparse Matrix)이란?

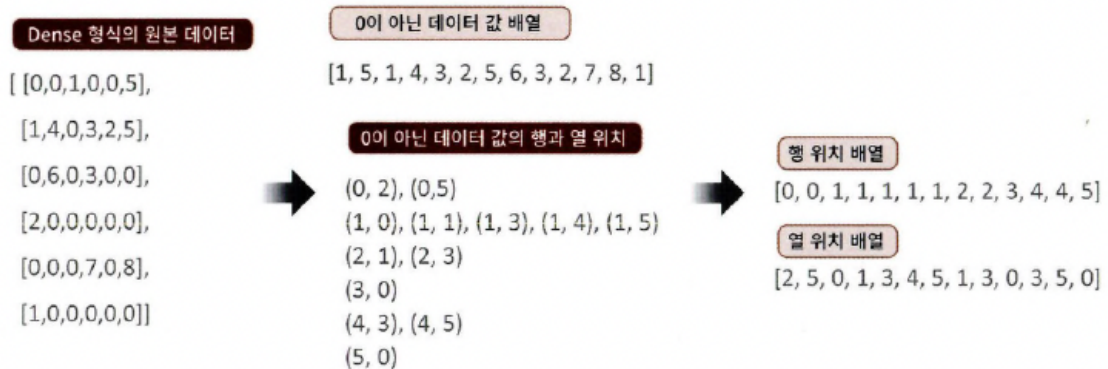
- 대부분의 값이 0으로 채워진 행렬

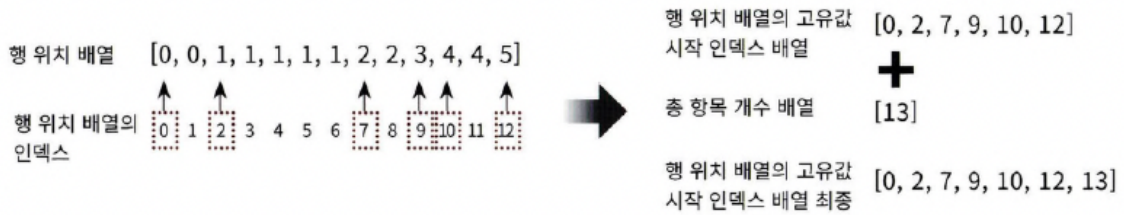
- 문서가 가진 단어 수는 적고 전체 단어 수(N)는 많아서 발생
- 희소 행렬의 형태

행렬 유형	설명
Dense Matrix	거의 모든 값이 존재 (0이 적음)
Sparse Matrix	대부분의 값이 0 (BOW 피처 벡터화의 전형적 결과)

희소 행렬의 저장 형식

- COO (Coordinate Format)
 - 0이 아닌 값만 저장하고, 그 위치(row, col) 저장
 - 예:
 - 값 배열: [3, 1, 2]
 - 행 위치: [0, 0, 1]
 - 열 위치: [0, 2, 1]
- CSR (Compressed Sparse Row)
 - COO에서 행 위치 반복 문제를 해결
 - 행의 시작 위치 인덱스만 따로 저장
 - 더 빠르고 메모리 효율적 → 기본 형식





✓ 5. 감성 분석이란?

- 문서의 감정, 의견, 기분, 태도 등의 주관적인 요소를 분석하는 기법.
- 예: 영화 리뷰, 상품 후기, SNS 게시물 등에서 긍정/부정 감성을 분류.
- 활용 분야: 소셜 미디어 모니터링, 여론 분석, 브랜드 평가 등.

📌 감성 분석의 방식

(1) 지도학습 기반

- 감성 레이블(긍정/부정)이 명시된 학습 데이터를 바탕으로 감성 예측 모델 학습.
- 일반적인 텍스트 이진 분류 문제로 간주할 수 있음.
- 사용 예시: IMDB 영화 리뷰 데이터를 이용한 감성 분석

(2) 비지도학습 기반

- Lexicon(감성 어휘 사전) 을 기반으로 단어의 긍정/부정 감성을 점수화하여 분석.
- 대표적인 사전: SentiWordNet, VADER, Pattern
- 감성 점수 계산 방식: 단어별 감성 점수의 합계로 문서의 전체 감성 판단.

📌 실습: 지도학습 기반 감성 분석(IMDB)

- 텍스트 전처리:
 - HTML 태그 및 특수문자 제거
 - 소문자 통일
- 피쳐 벡터화:
 - CountVectorizer 또는 TfidfVectorizer 사용
 - `ngram_range=(1,2)` , `stop_words='english'` 설정
- 분류 모델: Logistic Regression

- 평가 지표: 정확도(Accuracy), ROC-AUC
- 결과:
 - Count 기반: 정확도 88.6%, ROC-AUC 0.9503
 - TF-IDF 기반: 정확도 89.3%, ROC-AUC 0.9598

비지도학습 기반 감성 분석

(1) SentiWordNet

- WordNet 기반의 시맨틱 어휘 사전
- 각 단어의 긍정 점수, 부정 점수, 객관성 점수를 제공
- 정확도: 약 66%, 재현율 약 71%로 다소 낮음

(2) VADER

- 소셜 미디어 감성 분석에 최적화된 사전
- 간단한 API (`SentimentIntensityAnalyzer`) 사용으로 손쉽게 적용 가능
- 각 문서에 대해 'compound 점수'를 기준으로 긍/부정 판별
- 정확도: 약 69.6%, 재현율: 약 85% (SentiWordNet보다 우수)

감성 분석 모델 비교

모델	정확도	정밀도	재현율
SentiWordNet	0.6613	0.6472	0.7091
VADER	0.6956	0.6491	0.8514
지도학습(LogReg)	0.8936	—	—